

AGENTIC AI

Week 5: AutoGen 0.4 Framework

A New Era of Multi-Agent AI Systems: Architecture, Components & Python Implementation

Covering: AutoGen Core | AgentChat | Semantic Kernel | Distributed Runtime | Middleware

Style: Academic & Formal | Audience: Intermediate Practitioners | Production-Ready GenAI Reference

SECTION I: THEORETICAL FOUNDATIONS

1. Introduction to AutoGen 0.4: A New Era of AI Agents

The landscape of AI agent frameworks has undergone significant evolution since the original AutoGen library was introduced by Microsoft Research. AutoGen 0.4 represents a fundamental architectural rethinking — a ground-up rewrite designed to address the limitations of its predecessor and provide a robust, production-ready foundation for building sophisticated multi-agent AI systems.

Whereas earlier agent frameworks, including AutoGen 0.2, were primarily designed around conversational back-and-forth between two agents with fixed interaction patterns, AutoGen 0.4 introduces a modular, asynchronous, and event-driven architecture capable of supporting complex, large-scale multi-agent topologies. This transition is not merely incremental — it constitutes a paradigm shift in how AI agents are defined, composed, and orchestrated.

OCR Correction Note

The source presentation contained OCR artifacts. Corrected terminology used throughout this document:

'O\Y' → AutoGen 0.4 (version number, rendered as O.4 in OCR)

'Minidar' → Middleware (the message routing and delivery layer)

'D-messaging' → Distributed Messaging (cross-process/cross-host agent communication)

Garbled 'VALA, IN 7 1006 from Langchain 21/2 Teams...' → Structured Output support (Pydantic models,

type-safe responses, validated against schema from LangChain/Teams integrations)

1.1 Historical Context: From AutoGen 0.2 to AutoGen 0.4

To appreciate the significance of AutoGen 0.4, it is instructive to understand what it replaces. AutoGen 0.2 was built around a dual-agent model: a UserProxyAgent (which could execute code on behalf of the user) and an AssistantAgent (powered by an LLM). While effective for many tasks, this architecture exhibited several critical limitations in production deployments:

- Tightly coupled agent logic and communication infrastructure made it difficult to swap underlying models or runtime environments.

- Sequential message passing precluded true parallel agent execution within a single workflow.
- The absence of a formal event model made it challenging to implement complex conditional workflows and error recovery patterns.
- Scalability to multiple agents required bespoke orchestration code that was not portable across different deployment targets.

AutoGen 0.4 resolves each of these limitations through a principled architectural decomposition, described in the following sections.

1.2 The AutoGen 0.4 Ecosystem: A Layered Architecture

AutoGen 0.4 is not a single monolithic library. It is structured as a layered ecosystem of three complementary packages, each serving a distinct architectural role:

Package	Primary Role	Audience	Key Abstraction
autogen-core	Low-level agent runtime, message routing, event loop	Framework builders, advanced users	RoutedAgent, AgentRuntime, MessageContext
autogen-agentchat	High-level conversational agent patterns, team orchestration	Application developers	AssistantAgent, RoundRobinGroupChat, SelectorGroupChat
autogen-ext	Ecosystem integrations (OpenAI, Azure, code execution, tools)	Integration engineers	OpenAIChatCompletionClient, CodeExecutorAgent

This layered design enforces a clean separation of concerns: the core runtime handles message delivery and agent lifecycle management without any knowledge of what agents do; the agentchat layer provides convenient, opinionated abstractions for common conversational patterns; and the ext package bridges the framework to real-world AI services and infrastructure.

2. AutoGen Core: The Foundational Runtime

AutoGen Core is the bedrock upon which the entire AutoGen 0.4 ecosystem is constructed. It implements the fundamental mechanisms for agent definition, message routing, lifecycle management, and runtime orchestration. Critically, it is designed to be entirely agnostic to the

specific capabilities or implementations of the agents it manages — this is its defining architectural characteristic.

2.1 The Agent Interaction Framework

At its core, AutoGen Core provides a managed runtime environment — the Agent Runtime — responsible for orchestrating all interactions between agents. The runtime implements an asynchronous, event-driven message-passing architecture, meaning that agents communicate by publishing and subscribing to typed messages rather than calling each other's methods directly.

This design choice has profound implications for system behaviour:

- **Loose Coupling:** Agents have no direct references to other agents. They communicate exclusively through the runtime's message bus, which means an agent's implementation can be changed, replaced, or moved to a different process without modifying any other agent.
- **Asynchronous Execution:** The runtime is built on Python's asyncio framework. Agents handle messages asynchronously, enabling concurrent processing without blocking the event loop while waiting for LLM responses, API calls, or other I/O operations.
- **Location Transparency:** From an agent's perspective, sending a message to another agent is identical regardless of whether that agent runs in the same process, a different process on the same machine, or a remote host across a network.

2.2 Agnostic to Agent Abilities

One of AutoGen Core's most important design principles is its complete agnosticism regarding what agents are capable of. The runtime does not prescribe whether an agent uses an LLM, executes code, queries a database, calls an external API, or performs any other action. It concerns itself exclusively with message delivery and agent lifecycle.

This agnosticism enables AutoGen Core to manage extraordinarily heterogeneous multi-agent systems — a single workflow might simultaneously include LLM-powered reasoning agents, deterministic rule-based agents, database query agents, and human-in-the-loop agents, all interacting through the same runtime infrastructure.

This design philosophy is broadly comparable to LangGraph's approach of providing a general-purpose graph execution engine, but AutoGen Core distinguishes itself with a stronger emphasis on the actor model for agent definition and a more sophisticated approach to distributed deployment.

2.3 Decoupled Logic: Separating Agent Intelligence from Message Delivery

The principle of decoupled logic is the architectural mechanism through which AutoGen Core achieves its agnosticism. The framework establishes a strict boundary between two concerns:

Concern	Responsibility	Implemented By	Changes Independently
Agent Logic	What the agent does when it receives a message: LLM calls, data processing, tool invocation, state updates	The agent class developer	Yes — agent behaviour can be updated without touching infrastructure
Message Delivery	How messages are transported from sender to recipient: serialisation, routing, retry, ordering guarantees	The AutoGen Core runtime	Yes — runtime can be swapped (in-process to distributed) without changing agent code

In practical terms, this means that an agent developer only needs to implement the `on_message` method (or its decorated equivalents) to define what their agent does. The entire infrastructure concern of how messages reach the agent is handled by the runtime — transparently and without requiring any agent-level code changes when the deployment topology changes.

3. Agent Definition in AutoGen Core

Defining an agent in AutoGen Core involves implementing a Python class that inherits from the `RoutedAgent` base class and declaring message handlers using Python decorators. This design leverages Python's type system and decorator protocol to create a clean, declarative interface for agent behaviour specification.

3.1 The Agent Identity: AgentId

Every agent in an AutoGen runtime is uniquely identified by an `AgentId` — a structured identifier consisting of two components:

- **Type (kind):** A string label identifying the agent's category or role within the system. Multiple agent instances can share the same type, enabling type-based message broadcasting.
- **Key (identifier):** A string that uniquely distinguishes one agent instance from others of the same type. Combined with type, it forms a globally unique identity within the runtime.

The AgentId serves as the addressing mechanism for message routing. When an agent sends a message to another agent, it specifies the recipient's AgentId. The runtime uses this identifier to locate the agent instance and deliver the message, regardless of where that agent is running.

3.2 Message Handlers: The @message_handler Decorator

The primary mechanism for defining agent behaviour in AutoGen Core is the @message_handler decorator. This decorator, applied to methods within a RoutedAgent subclass, registers those methods as handlers for specific message types. When the runtime delivers a message of the corresponding type to the agent, the decorated method is automatically invoked.

The decorator-based approach provides several important benefits:

- **Type safety:** Each handler is registered for a specific message type, making message handling patterns explicit and verifiable.
- **Extensibility:** Multiple handlers can be defined within a single agent class, one per message type the agent supports.
- **Discoverability:** The set of message types an agent handles is immediately apparent from the class definition.

3.3 The on_message Lifecycle Method

Internally, the RoutedAgent base class implements an on_message method that serves as the universal message entry point. When the runtime delivers any message to an agent, on_message is invoked first. The RoutedAgent implementation then inspects the message's type and dispatches it to the appropriate @message_handler decorated method.

This two-level dispatch pattern — runtime to on_message to specific handler — provides a clean extension point for cross-cutting concerns: logging, authentication, rate limiting, or any other

pre/post-processing logic can be inserted at the `on_message` level without modifying individual handler methods.

4. Middleware: The Message Routing and Processing Layer

In the AutoGen 0.4 architecture, the Middleware layer (OCR-rendered as 'Minidar' in the source presentation) constitutes the infrastructure responsible for intercepting, processing, and routing messages as they transit between agents through the runtime. The Middleware layer effectively eliminates the need for agents to implement their own message routing logic, enabling clean separation between application-level agent behaviour and infrastructure-level message management.

4.1 What Middleware Does

Middleware in AutoGen Core sits between the message bus and the individual agent handlers. Each middleware component in the pipeline receives an incoming message, can inspect and transform it, and then either forwards it to the next middleware component or to the target agent's handler. This chain-of-responsibility pattern enables composable, reusable cross-cutting functionality:

- **Message Filtering:** Middleware can evaluate whether a message should be delivered to a specific agent based on content, source, or metadata criteria, without requiring the agent itself to implement filtering logic.
- **Message Transformation:** Middleware can reformat, enrich, or summarise messages in transit — for example, compressing long conversation histories before delivery to agents with limited context windows.
- **Observability:** Logging, tracing, and monitoring middleware can record all inter-agent messages, providing comprehensive audit trails for debugging and compliance.
- **Rate Limiting and Throttling:** Middleware can enforce per-agent or system-wide message throughput limits, protecting downstream services from overload.

4.2 Human-in-the-Loop via Middleware

A particularly significant application of the Middleware layer is the implementation of human-in-the-loop interactions. In AutoGen 0.4, the Middleware layer can intercept agent messages and present them to a human operator for review or approval before they are delivered to the target

agent. This design is architecturally clean: the agents themselves are unaware of whether human review is occurring — the Middleware layer handles the interaction transparently.

This is a substantial improvement over the AutoGen 0.2 approach, which required explicit `UserProxyAgent` instances to handle human interactions, making human-in-the-loop a first-class architectural concern rather than a transparent middleware capability.

5. Key Features of AutoGen 0.4

AutoGen 0.4 introduces several features that collectively elevate it from a research prototype to a production-grade framework for enterprise-scale multi-agent AI systems. The following sections detail the three principal feature categories.

5.1 Multi-Model Support

AutoGen 0.4 introduces a formally specified, extensible interface for integrating diverse AI models into the same agentic system. Rather than coupling agent behaviour to any specific LLM provider or API, the framework defines a `ChatCompletionClient` abstraction that can be implemented for any model provider.

This architecture enables several practically significant scenarios:

- **Model Specialisation:** Different agents within the same workflow can use different LLMs. An orchestrator agent might use a powerful but expensive model (GPT-4o) for high-level planning, while worker agents use faster, cheaper models (GPT-4o-mini, Claude Haiku) for routine sub-tasks.
- **Provider Diversity:** A single workflow can simultaneously incorporate agents backed by OpenAI models, Azure OpenAI endpoints, Anthropic Claude, Google Gemini, or locally hosted open-source models via Ollama, without architectural modification.
- **A/B Testing and Experimentation:** Model clients can be swapped at agent instantiation time, enabling systematic comparison of different LLMs on the same task without modifying agent logic.
- **Cost Optimisation:** Routing strategies can dynamically select models based on task complexity and cost thresholds, optimising the cost-quality tradeoff across a workflow.

5.2 Structured Output

Structured Output is one of the most practically valuable features introduced in AutoGen 0.4. It enables agents to produce responses that are guaranteed to conform to a specified data schema, eliminating the fragile string parsing that characterised earlier approaches to structured data extraction from LLMs.

The implementation leverages Pydantic models to define the expected output schema. When an agent is configured with a structured output schema, the framework automatically constructs the appropriate JSON schema for the LLM's structured output mode, validates the response against the schema, and surfaces any validation errors before they propagate to downstream agents.

The practical benefits for production systems are substantial:

- Downstream agents can reliably access structured fields from upstream outputs without defensive parsing code.
- Workflow validation logic can inspect intermediate outputs programmatically against business rules.
- Type-safe inter-agent communication reduces a major class of runtime errors in complex workflows.
- Integration with enterprise systems (databases, APIs, message queues) is simplified when agent outputs are structurally guaranteed.

5.3 Distributed Messaging

Distributed Messaging (OCR-rendered as 'D-messaging') is the capability that enables AutoGen 0.4 agents to communicate across process boundaries and network hosts. This feature is foundational for deploying multi-agent systems at enterprise scale, where agents may run on different machines, cloud services, or geographic regions.

The Distributed Messaging infrastructure in AutoGen 0.4 supports two primary runtime modes:

- **SingleThreadedAgentRuntime:** All agents execute within a single Python process and event loop. Messages are delivered in-memory without serialisation. Appropriate for development, testing, and single-machine deployments.
- **WorkerAgentRuntime + GrpcWorkerAgentRuntimeHost:** Agents are distributed across multiple processes or machines, communicating via gRPC. The runtime host acts

as a message broker, routing messages between distributed workers. This is the basis for the Distributed Runtime experimental feature.

⚠ **Distributed Runtime: Experimental Status**

As explicitly noted in the AutoGen 0.4 documentation: 'Distributed Runtime is regarded as experimental by Microsoft.'

This designation means the Distributed Runtime API may change between releases, and it has not yet been

subjected to the full production validation applied to stable features. It should be used in production

deployments only with appropriate risk assessment and testing coverage.

6. AutoGen AgentChat: High-Level Conversational Patterns

While AutoGen Core provides maximum flexibility and control, most application developers working on conversational AI systems will primarily interact with the autogen-agentchat package. AgentChat is built on top of AutoGen Core and provides a set of opinionated, high-level abstractions for the most common multi-agent conversational patterns.

6.1 The AssistantAgent

The AssistantAgent is the primary building block of the AgentChat layer. It encapsulates a complete LLM-powered conversational agent with the following integrated capabilities:

- **Model Client Integration:** A configured ChatCompletionClient instance that handles all LLM API communication, including authentication, retry logic, and response parsing.
- **System Message (Persona):** A configurable system_message that defines the agent's role, capabilities, communication style, and operational constraints — the agent's persona, as discussed in Chapter 1.
- **Streaming Support:** The model_client_stream=True parameter enables token-level streaming of LLM responses, reducing perceived latency for interactive applications and enabling progressive response display.
- **Tool Integration:** AssistantAgent natively supports tool/function calling, enabling agents to invoke external functions, APIs, or services as part of their response generation process.

6.2 Message Objects in AgentChat

All communication in AgentChat is mediated by typed Message objects. The `TextMessage` class is the most fundamental message type, carrying string content between agents. The architecture supports multiple message types for different communication purposes:

Message Type	Content	Typical Use
<code>TextMessage</code>	Plain text string content	Standard agent-to-agent conversational exchange
<code>MultiModalMessage</code>	Text combined with images or other media	Vision-capable agents processing visual inputs
<code>ToolCallMessage</code>	Structured tool invocation request	Agent requesting execution of a registered function
<code>ToolCallResultMessage</code>	Structured tool execution result	Returning function output to the requesting agent
<code>StopMessage</code>	Termination signal	Signalling workflow completion to team orchestrators

6.3 Team Orchestration Patterns

AgentChat provides several pre-built team orchestration patterns that implement common multi-agent coordination strategies without requiring manual orchestration code:

- **RoundRobinGroupChat:** Agents take turns responding in a fixed circular sequence. Appropriate for workflows where each agent contributes a distinct perspective in a predetermined order.
- **SelectorGroupChat:** A designated selector agent (typically LLM-powered) dynamically chooses which agent should respond next based on the conversation context. Appropriate for workflows requiring intelligent routing of complex, heterogeneous tasks.
- **Swarm:** Agents autonomously decide whether to speak based on their own assessment of relevance, without a central coordinator. Appropriate for exploratory, open-ended collaborative reasoning tasks.

7. Semantic Kernel: The AI Integration Bridge

Semantic Kernel is an open-source Software Development Kit (SDK) created and maintained by Microsoft that serves a complementary but distinct role from AutoGen in the AI application landscape. Where AutoGen focuses on multi-agent orchestration, Semantic Kernel focuses on the integration of AI capabilities into existing applications and services.

7.1 Core Purpose and Design Philosophy

Semantic Kernel is designed as a middleware bridge — it acts as an intermediary layer between AI model providers (primarily Azure OpenAI and OpenAI) and an organisation's existing codebase, data stores, and business logic. The framework provides a unified programming model for AI capability integration that abstracts away the specifics of individual model APIs.

The architectural metaphor of a 'kernel' is intentional: just as an operating system kernel mediates between applications and hardware, Semantic Kernel mediates between AI-powered applications and the underlying language models and services they depend upon.

7.2 Key Semantic Kernel Concepts

Semantic Kernel organises AI capabilities around several core abstractions:

- **Plugins (formerly Skills):** Collections of related AI functions that encapsulate specific capabilities. A plugin might provide functions for text summarisation, code generation, sentiment analysis, or domain-specific tasks. Plugins can be implemented as Semantic Functions (prompt-based) or Native Functions (Python/C# code).
- **Planners:** LLM-powered components that automatically construct sequences of function calls (plans) to achieve a specified goal, selecting and chaining appropriate plugin functions based on their semantic descriptions.
- **Memory:** An integrated vector memory system that enables semantic search over stored text, supporting the construction of RAG (Retrieval-Augmented Generation) pipelines with minimal boilerplate code.
- **Connectors:** Pre-built integrations with AI services including Azure OpenAI, OpenAI, Hugging Face, and vector databases such as Azure AI Search, Qdrant, and Chroma.

7.3 AutoGen vs. Semantic Kernel: Complementary Roles

A common source of confusion is the relationship between AutoGen and Semantic Kernel. They are not competing alternatives — they are complementary frameworks that address different layers of the AI application stack:

Dimension	AutoGen 0.4	Semantic Kernel
-----------	-------------	-----------------

Primary Focus	Multi-agent orchestration and communication	AI capability integration into applications
Core Abstraction	Agents communicating via message-passing	Plugins (functions) invoked by a kernel
Planning	Agent-driven, emergent from conversation	Explicit planner constructs function call sequences
Memory	External (not built-in at core level)	Built-in vector memory with semantic search
Human Interaction	Middleware-level, transparent to agents	Explicit user turn patterns
Best For	Complex multi-agent workflows, dynamic task decomposition	Integrating AI into existing .NET/Python applications
Relationship	Can use Semantic Kernel plugins as agent tools	Can invoke AutoGen agents as SK plugins

In practice, many production systems combine both frameworks: Semantic Kernel provides the AI capability integration layer (plugins, memory, connectors), while AutoGen orchestrates complex multi-agent workflows that leverage those capabilities.

8. The Agent Framework Landscape: Positioning AutoGen 0.4

AutoGen 0.4 exists within a growing ecosystem of agent orchestration frameworks. Understanding its positioning relative to peers is essential for making informed architectural decisions in production projects.

Framework	Publisher	Primary Paradigm	Strengths	Best Use Case
AutoGen 0.4	Microsoft Research	Actor model, event-driven	Distributed runtime, multi-agent, async	Enterprise multi-agent systems, distributed deployments
LangGraph	LangChain	Graph-based state machine	Fine-grained control, cyclical graphs, streaming	Complex conditional workflows with state management
CrewAI	CrewAI Inc.	Role-based agent crews	Simple API, role definition, task assignment	Rapid prototyping, team-based agent coordination
Semantic Kernel	Microsoft	Plugin/kernel integration	Enterprise integration, .NET support, memory	AI capability integration into existing applications
LangChain (Agents)	LangChain	Chain + tool calling	Vast ecosystem, community support	Tool-heavy single-agent applications

SECTION II: PRACTICAL IMPLEMENTATION & CODE

This section provides comprehensive, annotated implementations of the AutoGen 0.4 concepts introduced in Section I. All code is written for Python 3.10+ using autogen-agentchat 0.4.x and autogen-core 0.4.x. Each notebook cell is presented with its complete code followed by a detailed explanation of every statement's purpose and behaviour.

Notebook Cell 1: Environment Setup and Installation

Code

```
# — Cell 1: Installation and Environment Setup

# Run these commands in your terminal or as shell cells in Jupyter:

# Install the AutoGen 0.4 packages
# pip install autogen-agentchat==0.4.9
# pip install autogen-ext[openai]==0.4.9
# pip install python-dotenv

import os
import asyncio
from dotenv import load_dotenv

# AutoGen AgentChat imports
from autogen_agentchat.agents import AssistantAgent
from autogen_agentchat.messages import TextMessage
from autogen_agentchat.teams import RoundRobinGroupChat, SelectorGroupChat
from autogen_agentchat.conditions import TextMentionTermination,
MaxMessageTermination
from autogen_agentchat.ui import Console

# AutoGen Core imports
from autogen_core import SingleThreadedAgentRuntime, AgentId, MessageContext
from autogen_core import RoutedAgent, message_handler
from autogen_core import TopicId, TypeSubscription

# Extension: OpenAI model client
from autogen_ext.models.openai import OpenAIChatCompletionClient

# Load environment variables (API keys stored in .env file)
load_dotenv()
OPENAI_API_KEY = os.getenv('OPENAI_API_KEY')

print('AutoGen 0.4 environment initialised successfully.')
```

Explanation

Lines 1-8 (Installation): AutoGen 0.4 is distributed across multiple pip packages. `autogen-agentchat` provides the high-level conversational abstractions; `autogen-ext[openai]` provides the OpenAI model client integration. The `[openai]` extras syntax installs the optional OpenAI-specific dependencies alongside the base extension package.

Lines 13-17 (AgentChat imports): `AssistantAgent` is the primary high-level agent class. `TextMessage` represents the most common inter-agent message type. The `teams` module provides pre-built orchestration patterns. The `conditions` module provides termination condition implementations — `TextMentionTermination` ends a conversation when a specified phrase appears; `MaxMessageTermination` ends it after a maximum number of messages.

Lines 20-22 (Core imports): `SingleThreadedAgentRuntime` is the standard in-process runtime — all agents run within one Python process. `AgentId` uniquely identifies an agent instance. `RoutedAgent` is the base class for custom low-level agents. The `message_handler` decorator registers methods as typed message handlers. `TopicId` and `TypeSubscription` support the publish-subscribe messaging pattern.

Line 25 (Model Client): `OpenAIChatCompletionClient` implements the `ChatCompletionClient` interface for OpenAI's API. This abstraction means the same `AssistantAgent` code works with any model client implementation — swap `OpenAIChatCompletionClient` for `AzureOpenAIChatCompletionClient` or `AnthropicChatCompletionClient` without changing agent logic.

Notebook Cell 2: Configuring the Model Client

Code

```
# — Cell 2: Model Client Configuration —————

# Configure the OpenAI model client
# This client is shared across multiple agents or used per-agent
model_client = OpenAIChatCompletionClient(
    model='gpt-4o-mini',          # Cost-efficient; swap to gpt-4o for complex
    tasks                         # tasks
    api_key=OPENAI_API_KEY,      # Moderate creativity for conversational agents
    temperature=0.7,             # Limit response length per turn
    max_tokens=1500,
)

# For Azure OpenAI, use the Azure-specific client:
# from autogen_ext.models.openai import AzureOpenAIChatCompletionClient
# azure_client = AzureOpenAIChatCompletionClient(
```



```
# azure_deployment='gpt-4o',
# azure_endpoint=os.getenv('AZURE_OPENAI_ENDPOINT'),
# api_version='2024-08-01-preview',
# api_key=os.getenv('AZURE_OPENAI_API_KEY'),
# )

# Verify the client is configured correctly (optional health check)
async def verify_client():
    response = await model_client.create(
        messages=[{'role': 'user', 'content': 'Reply with: OK'}],
    )
    return response.choices[0].message.content

# Uncomment to run the verification:
# result = asyncio.run(verify_client())
# print(f'Model client status: {result}')

print(f'Model client configured: {model_client.model}')
```

Explanation

Lines 5-9 (OpenAIChatCompletionClient): The model client object encapsulates all configuration for LLM API communication. The model parameter specifies which OpenAI model to use. gpt-4o-mini is recommended for development due to its cost efficiency; production deployments may use gpt-4o for higher reasoning quality. The temperature parameter controls output randomness (0.0 = deterministic, 1.0 = highly creative); 0.7 balances coherence with variability for conversational tasks.

Lines 12-18 (Azure alternative): The commented Azure configuration demonstrates the multi-model support principle: switching from OpenAI to Azure OpenAI requires only changing the client class and providing Azure-specific credentials — no agent code changes are required. This is the model abstraction layer in action.

Lines 21-25 (Health check): The verify_client coroutine directly invokes the model client's create method, bypassing the agent layer. This is useful for confirming API key validity and model availability before committing to a full agent workflow execution. Note the use of async def — the AutoGen 0.4 model client API is fully asynchronous, built on Python's asyncio framework.

Notebook Cell 3: Creating AssistantAgents

Code

```
# — Cell 3: AssistantAgent Creation —————

# Agent 1: Content Planner — generates structured outlines
outline_agent = AssistantAgent(
```

```

    name='outline_agent',                # Unique name within the team
    model_client=model_client,           # Shared model client
    system_message=(
        'You are a senior content strategist. '
        'When given a topic, produce a structured, detailed outline '
        'with exactly 4 main sections and 3 sub-points each. '
        'Format as a numbered outline. Respond with the outline only.'
    ),
    model_client_stream=True,            # Enable token streaming
)

# Agent 2: Technical Writer — expands outlines into prose
writer_agent = AssistantAgent(
    name='writer_agent',
    model_client=model_client,
    system_message=(
        'You are an expert technical writer. '
        'You receive a structured outline and expand each section '
        'into comprehensive, academic prose paragraphs. '
        'Write at least 100 words per section. '
        'Use formal, precise language appropriate for a technical reference.'
    ),
    model_client_stream=True,
)

# Agent 3: Quality Reviewer — evaluates and critiques content
reviewer_agent = AssistantAgent(
    name='reviewer_agent',
    model_client=model_client,
    system_message=(
        'You are a senior editor and quality reviewer. '
        'Evaluate the provided technical content against these criteria: '
        '(1) accuracy and completeness, '
        '(2) clarity and readability, '
        '(3) appropriate academic tone. '
        'Provide specific, actionable feedback. '
        'When the content meets all criteria, respond with: APPROVED'
    ),
    model_client_stream=True,
)

print('Three AssistantAgents created successfully:')
for agent in [outline_agent, writer_agent, reviewer_agent]:
    print(f'  - {agent.name}')

```

Explanation

The name Parameter: Each agent's name must be unique within a team or group chat. Names are used for identification in conversation logs, routing decisions in SelectorGroupChat, and debugging output. Use descriptive, snake_case names that convey the agent's function.

The system_message Parameter: This is the agent's persona, as described in Chapter 1's theoretical framework. It is sent as the system role in every LLM API call the agent makes,

establishing its identity, behavioural constraints, and output format expectations. Note the multi-line string construction using implicit string concatenation — parentheses allow the string to span multiple source lines without backslash continuation.

model_client_stream=True: When enabled, the agent requests that the LLM stream its response token-by-token rather than waiting for the complete response before returning. This reduces perceived latency from the user's perspective (text appears progressively) and enables early termination if the response is found to be satisfactory partway through generation.

The APPROVED Termination Keyword: The reviewer_agent's system_message includes a specific termination keyword ('APPROVED') that will be detected by the TextMentionTermination condition in Cell 4. This design pattern — embedding structured signals in agent output — is a common AutoGen convention for workflow control.

Notebook Cell 4: Assembling and Running a Team

Code

```
# — Cell 4: Team Assembly and Execution

# Define termination conditions
# The team stops when the reviewer says 'APPROVED' OR after 15 messages
termination = (
    TextMentionTermination('APPROVED') # Keyword-based stop
    | MaxMessageTermination(15)         # Safety limit: prevent infinite
loops
)

# Assemble a RoundRobinGroupChat team
# Agents take turns in the order specified
content_team = RoundRobinGroupChat(
    participants=[outline_agent, writer_agent, reviewer_agent],
    termination_condition=termination,
)

# Define the workflow execution coroutine
async def run_content_workflow(topic: str) -> None:
    """
    Executes the content creation workflow:
    outline_agent -> writer_agent -> reviewer_agent -> (repeat or stop)
    """
    print(f'Starting content workflow for topic: "{topic}"')
    print('=' * 60)

    # Run the team and stream output to console
    result = await Console(content_team.run_stream(
```

```
        task=topic,
    ))

    print('=' * 60)
    print(f'Workflow completed. Stop reason: {result.stop_reason}')
    print(f'Total messages exchanged: {len(result.messages)}')
    return result

# Execute the workflow
# AutoGen 0.4 is fully async; use asyncio.run() in scripts,
# or await directly in Jupyter notebooks
topic = 'The Architecture of Transformer-Based Large Language Models'

# In a Jupyter notebook (already in async context):
result = await run_content_workflow(topic)

# In a Python script (synchronous context):
# result = asyncio.run(run_content_workflow(topic))
```

Explanation

Lines 5-7 (Termination Conditions): AutoGen 0.4 uses composable termination conditions. `TextMentionTermination('APPROVED')` scans each message for the specified keyword and terminates the team conversation when found. `MaxMessageTermination(15)` provides an absolute safety ceiling preventing infinite agent loops. The `|` operator composes them with OR semantics — the team stops when either condition is satisfied first.

Lines 11-14 (RoundRobinGroupChat): `RoundRobinGroupChat` implements the simplest orchestration pattern: agents take turns in the order specified in the participants list. After the initial task message, `outline_agent` responds first, then `writer_agent`, then `reviewer_agent`, and the cycle repeats. This predictable turn-taking makes conversation flow easy to reason about.

Lines 26-28 (Console + run_stream): The `run_stream` method returns an asynchronous generator that yields message events as they are produced. The `Console` utility consumes this stream and renders it to the terminal in real-time, including agent names and streaming token output. This is the recommended way to observe team execution during development.

Lines 40-44 (Asyncio execution modes): AutoGen 0.4 is built on Python's `asyncio` framework throughout. In Jupyter notebooks, the event loop is already running (maintained by IPython's kernel), so `await` syntax works directly. In standalone Python scripts, `asyncio.run()` creates and manages the event loop. Both execution modes produce identical results.

Notebook Cell 5: Working with Message Objects Directly

Code

```
# — Cell 5: Direct Message Object Usage —————

from autogen_agentchat.messages import TextMessage
from autogen_agentchat.base import TaskResult

# 5a. Creating a TextMessage object manually
# TextMessage is the standard inter-agent message type
user_message = TextMessage(
    content='Explain the concept of attention mechanisms in transformers.',
    source='user', # Identifies the message sender
)

print(f'Message content : {user_message.content[:60]}...')
print(f'Message source  : {user_message.source}')
print(f'Message type    : {type(user_message).__name__}')

# 5b. Sending a message directly to a single agent (not a team)
async def query_single_agent(agent: AssistantAgent, query: str) -> str:
    """Send a single message to an agent and retrieve its response."""
    response = await agent.on_messages(
        messages=[TextMessage(content=query, source='user')],
        cancellation_token=None,
    )
    # The response is a Response object; extract the chat message
    return response.chat_message.content

# 5c. Example: Direct single-agent query
query = 'What are the three key components of the Transformer architecture?'
answer = await query_single_agent(outline_agent, query)
print(f'Direct query response:\n{answer[:400]}...')

# 5d. Inspecting message history after a team run
if 'result' in dir(): # Check if result exists from Cell 4
    print(f'\nConversation history from team run:')
    print(f'Total messages: {len(result.messages)}')
    for i, msg in enumerate(result.messages[:3]): # Show first 3
        print(f' [{i}] {msg.source}: {str(msg.content)[:100]}...')
```

Explanation

Lines 8-10 (TextMessage construction): TextMessage is a Pydantic model — a structured data class with validated fields. The content parameter carries the message payload (a Python string for TextMessage). The source parameter identifies the sender; for user-initiated messages, 'user' is the convention. For agent-generated messages, the agent's name is used automatically.

Lines 17-24 (on_messages): The on_messages method is the primary single-turn interface for an AssistantAgent. It accepts a list of TextMessage objects (the current conversation context) and returns a Response object. The cancellation_token parameter accepts an asyncio

CancellationToken for graceful task cancellation — passing None disables cancellation. In production code, always provide a cancellation token to enable graceful shutdown.

Line 23 (response.chat_message): The Response object wraps the agent's output with metadata. chat_message contains the primary TextMessage response from the agent. The Response may also contain inner_messages — these are intermediate tool call messages and results generated during the agent's processing, providing visibility into the agent's internal reasoning steps.

Lines 31-35 (Message history inspection): The TaskResult returned by team.run_stream contains the complete messages list — every TextMessage exchanged during the team conversation in chronological order. Inspecting this list is essential for debugging multi-agent workflows, understanding where quality issues arise, and auditing agent behaviour in production systems.

Notebook Cell 6: Custom Agent with AutoGen Core

Code

```
# — Cell 6: Custom Low-Level Agent using AutoGen Core

# This demonstrates the actor model pattern from AutoGen Core,
# showing how agents are defined at the foundational layer.

from dataclasses import dataclass
from autogen_core import RoutedAgent, message_handler, MessageContext
from autogen_core import SingleThreadedAgentRuntime, AgentId

# 6a. Define custom message types as dataclasses
@dataclass
class ResearchRequest:
    topic: str
    depth: str = 'comprehensive' # 'brief' | 'comprehensive' | 'expert'

@dataclass
class ResearchReport:
    topic: str
    summary: str
    key_findings: list
    confidence_score: float

# 6b. Define a custom agent using the RoutedAgent base class
class ResearchAgent(RoutedAgent):
    """
    A custom research agent that processes ResearchRequest messages
    and produces ResearchReport messages.
    Built on AutoGen Core's actor model for maximum flexibility.
    """
```

```

"""

def __init__(self, model_client) -> None:
    super().__init__('Research Agent')
    self._model_client = model_client
    self._request_count = 0    # Internal state

    @message_handler
    async def handle_research_request(
        self,
        message: ResearchRequest,
        ctx: MessageContext,
    ) -> ResearchReport:
        """
        Handler for ResearchRequest messages.
        The @message_handler decorator registers this method
        for the ResearchRequest message type automatically.
        """
        self._request_count += 1
        print(f'[ResearchAgent] Processing request #{self._request_count}:
{message.topic}')

        # Construct the research prompt
        prompt = (
            f'Conduct a {message.depth} research analysis on:
{message.topic}\n\n'
            'Provide:\n'
            '1. A concise 2-sentence summary\n'
            '2. Three key findings as bullet points\n'
            '3. A confidence score from 0.0 to 1.0 on the research quality\n'
            'Format: SUMMARY: ... | FINDINGS: ... | CONFIDENCE: ...'
        )

        # Invoke the LLM
        response = await self._model_client.create(
            messages=[{'role': 'user', 'content': prompt}]
        )
        raw_output = response.choices[0].message.content

        # Parse structured output (simplified parsing for demonstration)
        parts = raw_output.split('|')
        summary = parts[0].replace('SUMMARY:', '').strip() if len(parts) >
0 else raw_output
        findings = parts[1].replace('FINDINGS:', '').strip().split('\n') if
len(parts) > 1 else []
        confidence = float(parts[2].replace('CONFIDENCE:', '').strip()) if
len(parts) > 2 else 0.7

        return ResearchReport(
            topic=message.topic,
            summary=summary,
            key_findings=[f.strip() for f in findings if f.strip()],
            confidence_score=min(max(confidence, 0.0), 1.0), # Clamp to [0,1]
        )

```

```
# 6c. Run the custom agent in a SingleThreadedAgentRuntime
async def run_custom_agent_demo():
    runtime = SingleThreadedAgentRuntime()

    # Register the agent type with the runtime
    await ResearchAgent.register(
        runtime,
        type='research_agent',
        factory=lambda: ResearchAgent(model_client),
    )

    # Start the runtime
    runtime.start()

    # Send a message to the agent via AgentId
    research_id = AgentId(type='research_agent', key='default')
    report = await runtime.send_message(
        ResearchRequest(topic='AutoGen 0.4 Architecture',
        depth='comprehensive'),
        recipient=research_id,
    )

    print(f'Research Topic      : {report.topic}')
    print(f'Summary              : {report.summary[:200]}...')
    print(f'Key Findings             : {len(report.key_findings)} items')
    print(f'Confidence Score         : {report.confidence_score:.2f}')

    await runtime.stop_when_idle()
    return report

report = await run_custom_agent_demo()
```

Explanation

Lines 10-18 (Dataclass message types): AutoGen Core uses Python dataclasses (or Pydantic models) as message type definitions. Each dataclass defines the schema of a specific message type. The type annotation on the `@message_handler` method parameter is what AutoGen Core uses to route messages to the correct handler — it inspects the type annotation at class definition time and registers the handler accordingly.

Lines 22-31 (RoutedAgent subclass): The `ResearchAgent` class extends `RoutedAgent` — the foundational base class for all AutoGen Core agents. The constructor calls `super().__init__('Research Agent')` to register the agent's display name with the runtime. The `_model_client` and `_request_count` instance variables demonstrate that agents can maintain private state across multiple message invocations — this is the actor model's state encapsulation principle.

Lines 33-36 (@message_handler): The @message_handler decorator is the core mechanism of AutoGen Core's routing system. It inspects the type annotation of the first parameter (message: ResearchRequest) and registers the method as the handler for ResearchRequest messages. Multiple @message_handler methods can be defined in one class, one per message type — each invoked only when a message of its registered type arrives.

Lines 73-76 (Agent registration): Agents in AutoGen Core are registered with the runtime via a factory function (the factory parameter). This factory is called when the runtime needs to create an agent instance. The lambda: ResearchAgent(model_client) closure captures the model_client from the surrounding scope. The type parameter ('research_agent') is the agent type string that forms part of its AgentId.

Lines 82-85 (send_message): The runtime's send_message method delivers a message to a specific agent identified by its AgentId. The AgentId combines type ('research_agent') and key ('default') to uniquely address an agent instance. When the runtime receives a send_message call, it locates the agent, invokes its on_message method (which dispatches to the correct @message_handler), and returns the handler's return value to the caller.

Notebook Cell 7: Structured Output with Pydantic

Code

```
# — Cell 7: Structured Output using Pydantic Models

from pydantic import BaseModel, Field
from typing import List, Literal

# 7a. Define a Pydantic schema for structured agent output
class TechAnalysisReport(BaseModel):
    """Structured output schema for technical analysis reports."""
    technology_name: str = Field(description='Name of the technology analysed')
    maturity_level: Literal['Emerging', 'Growing', 'Mature', 'Declining']
    summary: str = Field(min_length=50, max_length=500)
    key_advantages: List[str] = Field(min_length=2, max_length=5)
    key_limitations: List[str] = Field(min_length=1, max_length=5)
    production_ready: bool
    recommended_use_cases: List[str] = Field(min_length=1, max_length=4)
    overall_score: float = Field(ge=0.0, le=10.0)

# 7b. Create an agent with structured output schema
analyst_agent = AssistantAgent(
    name='tech_analyst',
    model_client=OpenAIChatCompletionClient(
```

```

        model='gpt-4o', # Structured output works best with GPT-4o
        api_key=OPENAI_API_KEY,
    ),
    system_message=(
        'You are a senior technology analyst. '
        'Analyse the given technology and provide a structured assessment. '
        'Be objective, evidence-based, and concise in all fields.'
    ),
    output_content_type=TechAnalysisReport, # Enforce structured output
)

# 7c. Invoke the agent and receive a validated Pydantic object
async def analyse_technology(tech_name: str) -> TechAnalysisReport:
    response = await analyst_agent.on_messages(
        messages=[TextMessage(
            content=f'Provide a comprehensive technical analysis of:
{tech_name}',
            source='user',
        )],
        cancellation_token=None,
    )
    # response.chat_message.content is now a TechAnalysisReport instance
    report: TechAnalysisReport = response.chat_message.content
    return report

# 7d. Execute and display structured result
tech_report = await analyse_technology('AutoGen 0.4 Multi-Agent Framework')

print('STRUCTURED TECHNOLOGY ANALYSIS REPORT')
print('=' * 50)
print(f'Technology      : {tech_report.technology_name}')
print(f'Maturity Level   : {tech_report.maturity_level}')
print(f'Production Ready: {tech_report.production_ready}')
print(f'Overall Score    : {tech_report.overall_score:.1f}/10.0')
print(f'Summary          : {tech_report.summary}')
print('Key Advantages:')
for adv in tech_report.key_advantages:
    print(f'  + {adv}')
print('Key Limitations:')
for lim in tech_report.key_limitations:
    print(f'  - {lim}')

# 7e. Validate the output programmatically
assert isinstance(tech_report, TechAnalysisReport), 'Type validation failed'
assert 0.0 <= tech_report.overall_score <= 10.0, 'Score out of range'
print('\nAll programmatic validations passed.')

```

Explanation

Lines 7-15 (Pydantic BaseModel): TechAnalysisReport is a Pydantic BaseModel subclass defining the expected output schema. Each field carries type annotations and validation constraints. Field(min_length=50, max_length=500) enforces string length bounds. Literal['Emerging', 'Growing', 'Mature', 'Declining'] constrains maturity_level to exactly four allowed

values. `Field(ge=0.0, le=10.0)` enforces numeric bounds on `overall_score` (`ge` = greater than or equal; `le` = less than or equal).

Line 28 (`output_content_type`): The `output_content_type` parameter is the key to AutoGen 0.4's structured output feature. When specified, it automatically configures the model client to request structured JSON output matching the Pydantic schema, parses the LLM's JSON response, and validates it against the schema — raising a validation error if the LLM produces a non-conforming response.

Line 43 (Type annotation on content): When `output_content_type` is configured, `response.chat_message.content` is not a string — it is an instantiated `TechAnalysisReport` Pydantic object. All field access is type-safe: `tech_report.overall_score` returns a float, `tech_report.production_ready` returns a bool, and so on. This eliminates the string parsing that would otherwise be required.

Lines 57-59 (Programmatic validation): The `assert` statements demonstrate downstream validation of the structured output. Because the content is a typed Pydantic object, `isinstance` checks and attribute-level assertions are reliable. This type safety is precisely what makes structured output valuable in production pipelines where downstream agents or services depend on specific field values.

Notebook Cell 8: SelectorGroupChat — Dynamic Agent Routing

Code

```
# — Cell 8: SelectorGroupChat with Dynamic Routing —————

# SelectorGroupChat uses an LLM to dynamically select which agent
# should respond next, based on conversation context.

from autogen_agentchat.teams import SelectorGroupChat

# Define specialised agents for different domains
ml_expert = AssistantAgent(
    name='ml_expert',
    model_client=model_client,
    system_message=(
        'You are an expert in Machine Learning and Deep Learning. '
        'Answer questions about algorithms, model architectures, training, '
        'and evaluation. Stay strictly within the ML/DL domain.'
    ),
)

data_engineer = AssistantAgent(
    name='data_engineer',
```

```

    model_client=model_client,
    system_message=(
        'You are a senior data engineer. Answer questions about data '
        'pipelines, ETL processes, data storage, streaming, and '
        'data infrastructure. Stay strictly within data engineering.'
    ),
)

cloud_architect = AssistantAgent(
    name='cloud_architect',
    model_client=model_client,
    system_message=(
        'You are a cloud solutions architect. Answer questions about '
        'cloud infrastructure, deployment, scalability, and DevOps. '
        'Stay strictly within cloud architecture and infrastructure.'
    ),
)

# Coordinator: manages the conversation flow
coordinator = AssistantAgent(
    name='coordinator',
    model_client=model_client,
    system_message=(
        'You are a technical coordinator. Summarise the expert responses '
        'received so far and determine if the user query has been fully '
        'addressed. '
        'If fully addressed, say COMPLETE. Otherwise, ask a clarifying '
        'question.'
    ),
)

# Assemble a SelectorGroupChat
# The selector_prompt guides the LLM in choosing the next speaker
expert_team = SelectorGroupChat(
    participants=[ml_expert, data_engineer, cloud_architect, coordinator],
    model_client=model_client, # The model client for the selector LLM
    termination_condition=TextMentionTermination('COMPLETE') |
    MaxMessageTermination(12),
    selector_prompt=(
        'You are a routing agent managing a panel of experts. '
        'Based on the conversation history, select the most appropriate '
        'participant to respond next. Consider: which expert is best suited '
        'to address the current question or gap in the conversation? '
        'Available participants: {participants}. '
        'Recent conversation: {history}. '
        'Respond with the name of the next participant only.'
    ),
)

# Execute with a multi-domain technical query
async def run_expert_panel(question: str):
    print(f'Expert Panel Query: {question}')
    print('-' * 55)
    result = await Console(expert_team.run_stream(task=question))

```

```
        return result

    panel_question = (
        'How should I architect a real-time ML inference system that '
        'handles 10,000 requests per second, processes streaming data, '
        'and deploys on AWS? What are the key components and trade-offs?'
    )
    result = await run_expert_panel(panel_question)
```

Explanation

Agent Specialisation Design: Each of the three specialist agents (`ml_expert`, `data_engineer`, `cloud_architect`) has a narrowly scoped `system_message` that explicitly constrains its domain. The instruction 'Stay strictly within [domain]' is critical — without explicit scope constraints, agents in a `SelectorGroupChat` tend to answer broadly rather than contributing their specialised expertise.

The SelectorGroupChat Mechanism: Unlike `RoundRobinGroupChat` (which rotates agents in a fixed sequence), `SelectorGroupChat` employs a dedicated LLM call — the selector — to dynamically choose which agent should speak next. After each agent's response, the selector receives the conversation history and the `selector_prompt`, reasons about which participant is most appropriate, and returns the name of the next speaker.

The selector_prompt Template: The `selector_prompt` is a template string with two special placeholders: `{participants}` is automatically replaced with a list of participant names, and `{history}` is replaced with recent conversation messages. This template is sent to the `model_client` on every selection decision, enabling the selector to make context-aware routing choices.

The coordinator Agent Role: The coordinator agent serves a dual function: it synthesises expert inputs into coherent summaries (ensuring the conversation converges toward an answer) and implements the termination signal by outputting 'COMPLETE'. This pattern — using a dedicated coordinator to manage workflow progression — is a common and effective design in `SelectorGroupChat` deployments.

Chapter Summary and Framework Reference

This chapter has provided a comprehensive treatment of the AutoGen 0.4 framework, covering its architectural foundations, core components, and practical implementation patterns. The following table consolidates the key concepts and their implementation counterparts:

Concept	Theory Section	Notebook Cell	Key AutoGen 0.4 API
AutoGen 0.4 Architecture	1.2	Cell 1	autogen-core, autogen-agentchat, autogen-ext
Model Client Configuration	5.1	Cell 2	OpenAIChatCompletionClient
AssistantAgent Creation	6.1	Cell 3	AssistantAgent(name, model_client, system_message)
Team Orchestration	6.3	Cell 4	RoundRobinGroupChat, termination_condition
Message Objects	6.2	Cell 5	TextMessage, on_messages(), TaskResult
Custom Core Agents	3.2, 3.3	Cell 6	RoutedAgent, @message_handler, AgentId
Structured Output	5.2	Cell 7	output_content_type, Pydantic BaseModel
Dynamic Routing	6.3	Cell 8	SelectorGroupChat, selector_prompt

Next Chapter Preview: Advanced AutoGen 0.4 Topics

Having mastered the AutoGen 0.4 foundations, the natural progression covers:

Memory Systems: Integrating vector stores (Qdrant, ChromaDB) for long-term agent memory.

Tool Integration: Registering Python functions, Semantic Kernel plugins, and REST APIs as agent tools.

Distributed Runtime: gRPC-based multi-process agent deployment across multiple machines.

Observability: Tracing with OpenTelemetry, logging with structured JSON, monitoring with Prometheus.

LangGraph vs AutoGen: Architectural decision guide for production multi-agent system selection.

-- End of Week 5: AutoGen 0.4 Framework --